

FLcompress: Lossless Data Compression Tool



Course: Design and Analysis of Algorithms (DAA) | **Algorithmic Paradigm:** Greedy Strategy (Huffman Coding)

Core Algorithm: Huffman Coding (Greedy)	Compression Ratio: ~2:1 (~50% size reduction)	Time Complexity: $O(N)$ Linear
Data Structures: Min-Heap, Binary Tree, Stacks	File Format: Custom packed .huff	Space Complexity: $O(N + K)$

1. Problem Statement

Modern computational systems process large volumes of uncompressed plain-text data (including program source code, logs, and ASCII/UTF-8 documents). Standard character representations allocate a fixed 8 bits (1 byte) for every character regardless of its actual frequency in the corpus. This results in significant storage redundancy and bandwidth overhead. The problem is to design an algorithmic solution that constructs an optimal, variable-length, lossless prefix encoding to minimize total bit consumption while providing exact and deterministic decompression.

2. Objective

- **Implement Greedy Huffman Coding:** Dynamically construct an optimal prefix-free binary tree where frequent characters receive shorter codes and rare characters receive longer codes.
- **Achieve Lossless ~50% Compression:** Reduce typical text file sizes by ~2:1 without dropping a single bit of information.
- **Self-Contained File Packaging:** Design a binary format (.huff) encapsulating the packed bitstream, post-order serialized tree metadata, and 64-bit length headers separated by unique delimiter markers.
- **Interactive UI & Verification:** Provide an easy-to-use Tkinter GUI and a comprehensive unit test suite validating edge cases.

3. Algorithm Explanation (DAA Greedy Strategy)

Huffman Coding relies on the **Greedy Choice Property** and **Optimal Substructure**. It builds an optimal prefix tree bottom-up by repeatedly merging the two subtrees with the lowest frequencies:

- **Step 1: Frequency Profiling:** Scan the input text of length N to count the frequency of each unique symbol K using a hash table $O(N)$.
- **Step 2: Priority Queue (Min-Heap):** Instantiate an $HNode$ for each character and insert into a min-heap prioritized by frequency $O(K \log K)$.
- **Step 3: Greedy Tree Construction:** Iteratively pop the two lowest-frequency nodes ($min1, min2$), merge them under a new internal parent with $freq = min1.freq + min2.freq$, and push parent back into min-heap. Repeat $K-1$ times until one root remains.
- **Step 4: Prefix Code Assignment:** Traverse from root to leaves (Left = '0', Right = '1') via DFS to generate prefix-free variable-length binary codes.
- **Step 5: Post-Order Tree Serialization:** Traverse the tree using two stacks to generate a compact post-order representation ('L' for leaves, 'B' for branches).
- **Step 6: Decoding & Reconstruction:** Reconstruct the tree via a single stack and traverse bit-by-bit from root to leaves to decode original symbols.

4. System Workflow & Architecture

Pipeline Stage	Compression Operations	Decompression Operations
1. File I/O & Validation	Validates file existence, non-emptiness, and UTF-8 plain-text type.	Validates file existence and confirms .huff extension.
2. Tree / Structure	Constructs Min-Heap & builds Huffman binary tree; serializes to post-order string.	Parses post-order serialized string via stack to rebuild exact tree structure.
3. Bit Manipulation	Encodes text to bit-string; packs bits into 8-bit bytes using <code>numpy.packbits</code> .	Unpacks raw bytes to bit-array using <code>numpy.unpackbits</code> up to original bit length.
4. Packaging	Concatenates [Code Bytes] + [Marker] + [64-bit Length] + [Marker] + [Serial Bytes] + [Marker].	Scans for 15-byte marker sequences (0xFF x 15) to isolate sections; decodes text to file.

5. Module Implementation Breakdown

- **flcompress_node.py (HNode):** Implements tree node representation storing symbol, frequency, parent, left, and right pointers. Overloads the comparison operator `__lt__` for automatic min-heap ordering based on character frequency.
- **flcompress_tree.py (HuffmanTree):** Core DAA engine. Manages `prioritize_nodes()` (frequency counting & min-heap insertion), `compress()` (greedy tree construction), `get_prefix_codes()` (recursive bit mapping), `serialize()` (two-stack post-order tree encoding), and `decompress()` (tree reconstruction & bit traversal).
- **compress_utilities.py (HuffFile):** Handles file validation (`_is_text_file`, `_validate_file`), NumPy bit-packing, 64-bit uint64 bit-length metadata, 15-byte delimiter sequences (`np.array([255]*15)`), and isolated subfolder management.
- **flcompressGUI.py:** Tkinter graphical interface styled with ttk widgets and Pillow (PIL). Features 'Compress a File' and 'Decompress a File' actions with automatic file size reduction reporting.
- **test_flcompress/test_flcompress.py:** Automated unittest suite evaluating small/large files, HTML files, 0-byte files, and invalid binary file types (.pdf, .jpg, .xls).

6. Complexity Analysis

Let **N** = total number of characters in file, **K** = number of distinct characters (alphabet size, $K \leq 256$ for ASCII).

Operation / Phase	Algorithm / Method	Time Complexity	Space Complexity
Frequency Analysis	Hash Map character count	$O(N)$	$O(K)$
Min-Heap Insertion	Push K character nodes into Priority Queue	$O(K \log K)$	$O(K)$
Huffman Tree Building	Greedy extraction & merging (K-1 merges)	$O(K \log K)$	$O(K)$
Prefix Code Traversal	DFS tree traversal to leaves	$O(K)$	$O(K)$
Encoding Bit-String	Map input string to prefix codes	$O(N)$	$O(N)$
Tree Serialization	Two-stack post-order traversal	$O(K)$	$O(K)$
Bit-Packing (NumPy)	Bit array to byte chunk conversion	$O(N)$	$O(N / 8)$
Total Compression	End-to-End Pipeline	$O(N + K \log K) \approx O(N)$	$O(N + K)$
Total Decompression	Tree Restore + Bit Traversal	$O(N + K) \approx O(N)$	$O(N + K)$

7. Experimental Results & Verification

Test Benchmark File	Format	Original Size	Compressed (.huff)	Reduction (%)	Lossless Integrity
test_small_file.txt	Plain Text	7,273 B (~7.1 KB)	3,852 B	47.0%	Exact Match (100%)
test_html_file.html	HTML Code	1,728 B (~1.7 KB)	982 B	43.2%	Exact Match (100%)
test_large_file.txt	Large Text Corpus	110,557 B (~108 KB)	57,412 B	48.1%	Exact Match (100%)

Automated Unit Test Execution (8/8 Passed):

- Ran 8 tests in 0.288s — OK
- Verifies round-trip decompression integrity across small, medium, and large text files.
 - Verifies error handling for 0-byte files and rejection of invalid binary formats (.pdf, .jpg, .xls).



8. Viva Questions & Quick-Reference Guide

Q1: Why is Huffman Coding classified as a Greedy algorithm?

At each step of tree construction, the algorithm makes a greedy local choice by combining the two nodes with the lowest frequencies from the priority queue. This local decision leads to a provably globally optimal prefix-code binary tree.

Q2: What is the significance of the prefix-free property?

No assigned codeword is a prefix of any other codeword because all characters reside exclusively at leaf nodes. This allows instantaneous, unambiguous decoding during decompression without any delimiter bits.

Q3: How is the Huffman tree stored inside the compressed file?

The tree is serialized using a two-stack post-order traversal into tokens ('L' + character for leaves, 'B' for branches). During decompression, a single stack reconstructs the exact tree in O(K) time.

9. Conclusion

The **FLcompress** project successfully demonstrates an end-to-end implementation of the **Huffman Coding** algorithm under the **Greedy algorithmic paradigm**. By combining min-heap priority queues, binary tree prefix traversal, two-stack post-order serialization, and NumPy byte packing, the tool achieves an average **~45–50% lossless compression ratio (~2:1)** with optimal **O(N) linear time performance** on plain-text files.

10. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms (4th ed.)*, Chapter 15: Greedy Algorithms — Huffman Codes. MIT Press.
2. Huffman, D. A. (1952). 'A Method for the Construction of Minimum-Redundancy Codes'. *Proceedings of the IRE*, 40(9), 1098–1101.
3. Python Standard Library Documentation: heapq — Heap queue algorithm module.